Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.0000000

Frontier and Localhost: A Survey of How Production AI Learns Outside the Weights

MIKHAIL L. ARBUZOV[1], LEE MOSBACKER[2], SISONG BEI[3], ZIWEI DONG[4], DMITRI KALAEV[5], AND ALEXEY SHVETS[6]

[1]Independent Researcher, CITY, COUNTRY (e-mail: mike.arbuzov54@gmail.com)
[2]Independent Researcher, CITY, COUNTRY (e-mail: lee.mosbacker@gmail.com)
[3]Independent Researcher, CITY, COUNTRY (e-mail: qurining@gmail.com)
[4]Independent Researcher, CITY, COUNTRY (e-mail: ziwei.dong@alumni.emory.edu)
[5]Independent Researcher, CITY, COUNTRY (e-mail: kalaevdr@gmail.com)
[6]Palo Alto Networks, CITY, COUNTRY (e-mail: ashvets@paloaltonetworks.com)

Corresponding author: Mikhail L. Arbuzov (e-mail: mike.arbuzov54@gmail.com).

TODO(authors): funding statement, e.g., "This work received no external funding." A. Shvets is employed by Palo Alto Networks; the views expressed are the authors' own.

ABSTRACT Production systems built on large language models now adapt mostly outside the model weights, and this article asks what optimization discipline that adaptation needs. After a failure, teams edit instructions, skills, memories, tools, orchestration graphs, evaluation suites, and governance rules far faster than any weight update. Two earlier results of this series explain why that layer is the right place: errors concentrate in a few key decisions that fail in a small, recurring catalogue of modes. We read the layer as one surface governed by one loop: a recurring failure is traced to a scaffold coordinate, one edit is proposed, a gate accepts or rejects it on measured loss, and a second decision sets how far the accepted edit may travel. Five conditions make this loop an optimizer rather than patchwork. The decisive one, a measurable loss, depends on the feedback signal the gate receives: an exact check (code, mathematics), an observable outcome (completed tasks), a judgment (writing, design), or a delayed reward (research). The signal's class fixes what the gate can be, which pairings of algorithmic and human gates are reachable, and which failure dominates. Across 256 production and research systems, scored on six substrates, with the scores released, the local loop is routine and the governed second loop has not been built: no surveyed system combines automated local adaptation with promotion across organizations under versioned lineage, access control, and rollback. We map the failure surface this gap leaves open and state the open problem each signal class poses.

INDEX TERMS AI governance, agent memory, agent skills, evaluation, large language models, LLM agents, prompt optimization, self-improving systems, survey

I. INTRODUCTION

The standard mental model of a system built on a large language model (LLM) is the model. Pre-training fixes the weights, fine-tuning nudges them, and deployment wraps a thin layer of prompts and tools around the result. Production in 2025 and 2026 has stopped behaving this way. Frontier weights change on a cycle of months; the systems built on them change daily, in instruction files checked into repositories, memories that persist across sessions, skills loaded on demand, registered tools, evaluation suites that block a merge, agent topologies, and pull-request gates on shared rule repositories. A growing share of the adaptation that makes a deployed system reliable now happens outside the weights, in a fast-changing local scaffold wrapped around a slow-changing frontier model.

Read one at a time, these artifacts look like unrelated engineering. A skill file is a procedure the model did not have. A repository rule is a local parameter for one project's conventions. A retrieval connector is memory access specific to one workflow. A tool registry provisions a capability the base model cannot reach. A pull-request review against a shared rule repository is a promotion gate, and an evaluation suite that blocks a merge is a validation check before promotion. Read together, they are one adaptation surface, the non-model part of what has been called a compound AI system [1]. The question this survey asks is what optimization discipline that surface needs. The answer we find is that the field has built an external adaptation layer for LLMs but not the optimizer

that should govern it. Most deployments maintain the layer as patchwork: a failure happens, someone adds a rule, the pile grows, nobody knows which addition helped, and fixes leak into contexts that never validated them.

The disciplined alternative runs two loops, and only the first is routine. The local loop, which turns a recurring failure into a tested edit, closes under an automatic gate in most research systems and a growing number of production ones. The governed second loop, which would carry a validated edit across organizations under versioned lineage, access control, and rollback, appears in none of the 256 systems surveyed. What separates them is the signal the gate receives. The class of that signal fixes which loop a deployment can close: an exact check or an observed outcome can gate an edit automatically, a judgment only once the judge is calibrated against people, and a delayed reward not edit by edit at all.

This article makes five contributions.

1. The loop that turns patchwork into optimization, in plain language and as a gated update rule, with the five conditions under which it is an optimizer.
2. Four classes of feedback signal, from an exact check to a delayed reward, each fixing what the gate can be, which gate pairings are reachable, and which failure dominates.
3. A survey of 256 production and research systems, scored on six substrates against a published rubric, with the scores released.
4. A design space indexed by the gate on each loop, read structurally, and the convergence of five research families on the same loop as evidence that the architecture is real.
5. A four-property audit criterion for governed cross-organization scaffold optimization, which no surveyed system meets, and the failure surface its absence leaves open.

Section II argues that adaptation belongs in the scaffold, Section III states the loop, and Section IV classifies its signal; Sections V to VIII report the survey, and Sections IX to XI name the missing architecture, its failure surface, and the open problems.

The survey runs no experiments. Its evidence is the published behavior of 256 systems, scored on a released table.

II. WHY ADAPTATION LIVES OUTSIDE THE WEIGHTS

A. BREADTH IN THE WEIGHTS, DEPTH IN THE SCAFFOLD

Frontier training optimizes for breadth. The weights are tuned to generalize across millions of unseen deployments, and the objective smooths over exactly what one deployment cares about: local naming conventions, contradictory preferences between customers, rare workflow-specific failures. Reinforcement learning from human feedback reduces output diversity and homogenizes preferred style across users [2], [3], and instruction-tuning data rewards patterns that generalize across the annotator pool, so the frontier model is trained to disfavor one customer's idiosyncrasies. Tiwari et al. [4] reach the same place from the optimization side: in-context adaptation is cheap but capacity-bounded, in-weights adaptation is expressive but induces forgetting and loss of plasticity [5], so a two-timescale system, one that learns slowly in the weights and fast outside them, needs a cheap outer layer that absorbs what context can carry.

Production reliability is achieved in depth, inside one deployment, and it turns on what global training had to smooth away. Encoding every deployment in the weights is ruled out three times over: fine-tuning per customer multiplies the training, evaluation, and serving lifecycle by the number of deployments; weights ship quarterly at best, while the deployment wants daily updates; and overfitting the weights to one customer degrades the neighbors whose preferences contradict. Per-customer adapters [6], distilled models, and learned routing could absorb some of the residual, the failures that remain after training, but in practice they are rare. Adaptation lands in the scaffold, the one substrate that is cheap, inspectable, reversible, scoped to one customer, and updatable daily.

B. WHY THE SCAFFOLD IS THE RIGHT ARCHITECTURE: THE TWO EARLIER RESULTS

The case for learning outside the weights does not rest on cost alone; it is the conclusion of the two papers that precede this one in the series. *Beyond Exponential Decay* [7] finds that long LLM outputs do not fail at the rate a uniform per-token error model predicts, because reliability hinges on a sparse set of key decisions, which make up under a tenth of tokens by the available long-context measurement. *The Architecture of Errors* [8] finds that inside a bounded deployment those decisions fail in a finite catalogue of recurring modes whose size grows only slowly with the number of observed failures, so a small, targeted budget of interventions covers the residual.

Together they say where a deployment's errors are and that they are few enough to fix one at a time. This article asks where the fixes live, and the corpus answers: in persistent artifacts outside the weights, where a fix for a recurring, deployment-specific mode is cheap to make, inspect, and reverse. Gated and scoped, that layer is not a workaround for an inadequate model. It is the architecture the error structure calls for: a frontier model that generalizes, a local scaffold that specializes, and reliability that comes from governing the specialization.

C. RELATION TO EXISTING SURVEYS

This survey classifies systems on four axes: whether a feedback loop closes at all, whether its gate is a program or a person, what the gate can observe, and whether promotion beyond one deployment is governed. No existing survey uses all four, and Table 1 places this one against the surveys that cover neighboring ground.

Existing surveys of self-evolving agents, agent memory, context engineering, skills, and harness design catalogue mechanisms by what they edit and when. None classifies systems by loop closure, gate type, or signal class; two carry a governance axis, and none covers promotion across organizations. One of those two, Ding et al.'s survey of persistent-state agents, finds that literature concentrated on accumulating

TABLE 1. Related surveys and the axes they classify by ("partial": discussed but not a classification axis).

Survey	Scope	Organizing axes	Loop closure	Gate type	Signal class	Governance
Gao et al. [9]	self-evolving agents	what, when, how, where to evolve	partial	partial	no	no
Fang et al. [10]	self-evolving agents	inputs, agent system, environment, optimizers	partial	no	no	no
Zhou et al. [11]	self-evolving coding agents (108 methods)	evolution target, timing, code signals	partial	partial	partial	partial (reversibility)
Ding et al. [12]	persistent-state agents (435 works)	authority, scope, mutability, provenance, recoverability	no	partial	no	yes (provenance, rollback)
Lin et al. [13]	memory security (~90 works)	lifecycle phases × security objectives	no	partial	no	yes (principal scoping, rollbackability)
Guo et al. [14]	agent system and harness design	six runtime responsibilities	partial	partial	partial	partial
Xia et al. [15]	skill-mediated agents (8 systems)	four responsibility layers, ten patterns	partial	no	no	partial
Ding et al. [16]	skill evaluation and evolution	four evolution paradigms × six benchmark classes	partial	partial	partial	partial
Mei et al. [17]	context engineering (1,411 papers)	components × system implementations	no	no	no	no
This survey	256 scaffold-adaptive systems	loop closure, gate type, signal class, governance	yes	yes	yes	yes (four-property audit)

and retrieving state rather than on governing, recovering, or relinquishing it [12].

A reader will expect the prompt-optimization lineage here: OPRO, EvoPrompt, Promptbreeder, DSPy, and TextGrad [18]–[22]. That lineage optimizes a prompt or program once against a development set and is not a deployed loop. The evolving playbook of agentic context engineering [23] persists its artifact but keeps it inside one context and never promotes it.

The closest precedents optimize the scaffold itself. Agent symbolic learning updates prompts, tools, and pipeline as symbolic parameters from textual feedback [24]; in STOP, a scaffolding program recursively improves its own improvement code [25]; and a coding agent edits its own harness to raise its benchmark score [26]. Each runs one deployment's loop under a benchmark score. What changes here is that the edited artifact persists in a deployment and must clear a second gate before it earns any scope beyond that deployment (Supplementary Material S7).

III. THE LOOP: ARTIFACT-LAYER DESCENT

A. THE LOOP IN PLAIN LANGUAGE

The disciplined version of scaffold maintenance is one loop, which we call artifact-layer descent, run again and again over the life of a deployment. It does not react to single failures. As the deployment runs, it logs what went wrong: failed outputs, user corrections, rejected answers, evaluation regressions, tool-call errors. Reflecting over that record, it clusters the failures into recurring modes and takes up the ones that matter most, ranked by how often each recurs and how badly it fails. For such a mode it locates the scaffold coordinate responsible, which may be a stale memory entry, an under-specified instruction, a missing tool argument, a brittle retrieval rule, or a misrouted agent edge. It then synthesizes one candidate edit to that coordinate. A deployment serves one **patch**: the slice of users, tasks, and conventions it handles. The failure rate on that patch's recurring tasks is the **patch loss**; the loop measures whether the edit lowers it, and a gate accepts or rejects the edit on that measurement. An accepted edit stays in the scaffold; one that a later check shows made things worse is pruned or rolled back.

Then comes the step most systems skip. The loop decides how far the edit should travel, promoting it to other deployments only as far as the evidence reaches; we call that limit the edit's **evidence radius**. The check that belongs at this step is **receiving-side evaluation**: a check run where the edit arrives, not where it was made. Fig. 1 draws the loop.

B. TWO LOOPS, TWO KNOBS

An engineer notices that a coding agent keeps misnaming a class in one repository. The fix is a one-line rule in that repository's instruction file. The edit is small and its scope is one project. The same engineer could promote the rule to a team-wide template, to the company's core instructions, or to a skill registry shared across companies. The edit did not change; the scope did. And the gate that should let it through

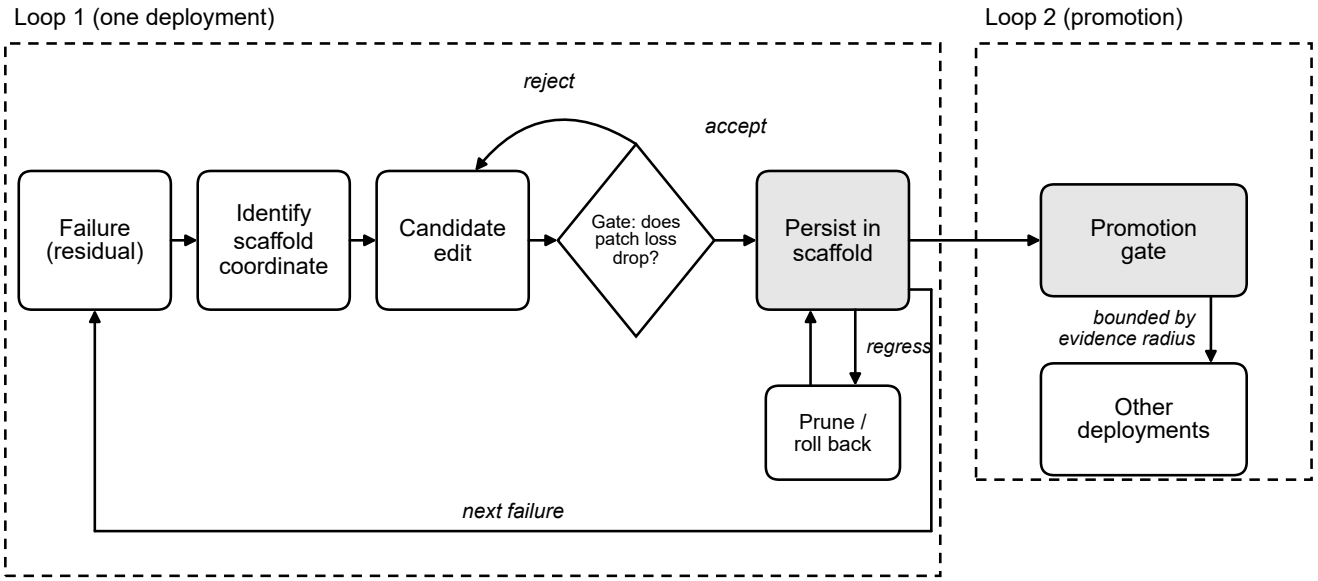


FIGURE 1. Artifact-layer descent. Loop 1 turns a failure into a gated, persisted edit on one deployment's scaffold; Loop 2 promotes a validated edit to other deployments, only as far as its evidence radius supports.

gets stricter at each step, because the loss it is tested against is now an average over more deployments, and a fix that helps one repository can hurt another.

Loop 1 is the local fix: a recurring failure produces a candidate edit, the edit is tested, and an accepted edit changes one deployment's scaffold. Loop 2 is the separate decision of whether that fix should be shared. In optimizer terms, Loop 1 sets the analogue of a step size: how much one accepted edit changes the local scaffold. Loop 2 sets how many deployments the edit applies to, which leaves the step alone and changes the target: the average failure rate the scaffold is tuned to reduce.

The scopes nest, USER within PROJECT within STACK within TENANT within CORE, and each level's loss is the average failure rate over the deployments it contains (Fig. 2). USER is one person's account or fork; PROJECT, one repository, workflow, or product workspace; STACK, a curated bundle of projects under one team; a **tenant** is one organization or customer; CORE is the scope every tenant shares. Promotion that reaches CORE crosses tenants, and we call it cross-tenant promotion; reaching TENANT applies an edit across one tenant's deployments. Governance is therefore not administrative overhead. It is the regularization that keeps a small fix from being aimed at the wrong target.

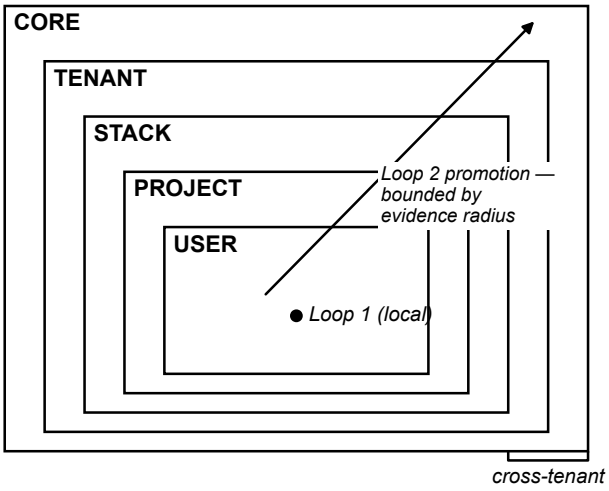


FIGURE 2. Patch scope hierarchy. Each level's loss averages the deployments it contains. Loop 1 descends a USER or PROJECT loss; Loop 2 promotes upward; cross-tenant promotion means reaching CORE.

C. WHEN THE LOOP IS AN OPTIMIZER

The loop is an optimization process, rather than a description of good practice, when five conditions hold:

1. a measurable patch loss, such as a failure rate, a correction rate, or an evaluation score;
2. captured residuals, so that failures are logged rather than lost;
3. credit assignment, so that a failure can be traced, at least sometimes, to the scaffold coordinate that caused it;
4. a synthesized candidate edit to that coordinate; and
5. a gate that checks whether the edit lowers the loss before the edit is kept or promoted.

The conditions presuppose frozen frontier weights, edits that persist as inspectable artifacts rather than transient prompts, and promotion that widens an artifact's scope under a gate rather than averaging comparable updates as weight-space federated learning does.

Write the patch loss of deployment D as

$$L_D(\theta_s) = \mathbb{E}_{x\sim D}[\ell(\pi(\theta_M, \theta_s, x))]\,,$$

where θ_M are the frontier weights, fixed during deployment, θ_s is the scaffold, a structured object over the six substrate coordinates of Section V, and ℓ is an operational loss. A failure yields no analytic gradient. It yields a candidate edit z_t, proposed by reflecting over the accumulated failure history, with modes taken in order of how often and how badly each fails. The gate accepts the edit when its noisy estimate of the change in loss clears a margin,

$$G_t(z_t) = 1 \iff \widehat{\Delta}_{z_t} L_D(\theta_s^t) + \lambda\,\Omega(z_t) \le -\tau_t,$$

where $\Omega \ge 0$ penalizes the complexity of the edit and the breadth of its promotion, and $\tau_t \ge 0$ is the required margin. An accepted edit is applied; a rejected one leaves the scaffold unchanged.

Production systems implement this inequality without writing it: an evaluation blocks a merge on regression, a reviewer rejects a vague rule, a governance system holds an artifact back until evidence accumulates. Under further assumptions on the gate's bias and the proposal process, the update satisfies a one-step descent inequality and converges, informally, to a gate-stable scaffold, where no admissible edit that would lower the loss clears the gate (Appendix XII). Where a condition fails, the picture is an analogy, and the survey marks which systems meet which conditions.

Each loop has a gate, and we name three kinds by who decides. A gate is **Auto** when a program makes the decision, whether by an evaluation score, a verifier, a reinforcement-learning threshold, a language-model judge [27] with a quantified agreement rate, or, at the degenerate end, an unconditional rule that accepts every candidate. It is **Human** when a person decides. It is **None** when the loop is absent. We write a system's two gates as a pair, Loop 1 first: [Auto × Human] means an algorithmic local gate and a human promotion gate. These pairs index the design space of Section VII.

Condition 1 says the loss must be measurable. It does not say what measures it, and that turns out to be the question that sorts the corpus.

D. THREE STAGES OF SCAFFOLD MAINTENANCE

Three stages of scaffold maintenance recur across the corpus (Table 2). They differ in how candidate updates are produced, whether credit assignment is explicit, and whether promotion is bounded by evidence. A patchwork loop becomes Stage 3, artifact-layer descent, once the five conditions hold. Production deployments that reach Stage 3 on the local loop promote either behind a human gate, as Spotify [28] and SkillForge do, or, in one case, by an automatic push that carries lineage but no evaluation, as Letta's Context Repositories do.

IV. WHERE THE SIGNAL COMES FROM: FOUR CLASSES OF FEEDBACK

Four scaffolds all run the loop of Section III. A coding agent changes one line in its repository rule file; the test suite runs and reports pass or fail within seconds. A scheduling assistant changes how it phrases a booking request; the appointment either appears on the calendar or it does not, and the system learns which an hour later. A marketing assistant changes the tone rule in its instructions; nobody can say whether the next draft is better, only whether a reviewer prefers it. A research assistant changes how it chooses which direction to pursue; whether the choice was good becomes visible months later, after many other edits have landed. All four produce a candidate edit z_t. They differ in what the gate G_t can know about it.

The gate accepts an edit when its estimate $\widehat{\Delta}_{z_t} L_D$ of the loss change clears a margin. Condition 1 asks for a measurable patch loss without saying what produces that estimate. We distinguish four classes of feedback signal by what produces it, and the class fixes three things at once: what the gate can be, which pairings of gate types a deployment can reach, and which failure dominates (Table 3).

C1, exact. A deterministic checker returns pass or fail, or an exact score, immediately and repeatably: unit tests, compilers, type checkers, exact-match answer keys, proof checkers, schema validators. DGM accepts a self-modification only after it passes in a sandbox on coding benchmarks [29]; AlphaEvolve scores candidate programs with a cascade of exact evaluators [30]; SkillOpt gates a skill edit on a held-out exact-match split of a math benchmark [31]. The estimator bias ε_t is close to zero, the margin τ_t can sit near zero, and accepting on a single evaluation is defensible, because repeating the check would return the same answer. Two things stay hard. A failing test localizes the output, not the scaffold coordinate that caused it, so credit assignment remains open; and a checker that is also the training target can be gamed. The dominant failures are test overfitting and scaffold bloat, because cheap acceptances accumulate.

C2, observable outcome. The environment or the user emits an outcome after an episode: a booked appointment, a closed ticket, a merged pull request, a tool call that returns without error, a retrieved document that contains the answer. The signal is real but one episode late, noisy, and confounded, since a user can accept a wrong answer and reject a right one. Gates of this class include Devin's merged pull requests [32], SAGE's task success on AppWorld [33], SkillRL's success-rate threshold [34], and PACE's failure signals from a deployed small model [35]. The gate becomes a reinforcement-learning threshold or a held-out task suite with a strictly positive margin, and the noise is absorbed by batching episodes or by requiring agreement across samples (Appendix XIII). Credit assignment takes its hardest form: the outcome says *that* the episode failed, not *why*. Most production deployments live in this class, and its signal arrives through a channel an attacker can control, which is why poisoned memory and tool-output injection dominate.

TABLE 2. Three stages of scaffold maintenance.

Stage	Update mechanism	Credit assignment	Promotion discipline	Dominant failure
1. Patchwork	Intuition-driven local edits	Implicit, ad hoc	None	Bloat
2. Random artifact search	Many variants tried under an evaluation	Weak; the evaluation is the only signal	Evaluation-gated merge, no failure-to-coordinate tracing	Overfitting to the in-distribution patch
3. Artifact-layer descent	Residual traced to a coordinate, then one edit	Explicit per failure mode	Bounded by evidence radius; reversible	Evaluation fragility

TABLE 3. The four feedback-signal classes and what each fixes.

Class	Signal	Gate G_t becomes	Conditions (1)–(5) of Sec. III satisfied	Gates the class permits (Loop 1 × Loop 2)	Failure that dominates (Sec. X)	Corpus examples
C1 exact	deterministic pass/fail or exact score, immediate	surrogate verifier; exact eval cascade; $\tau_t \approx 0$	(1)(2)(4)(5) directly; (3) credit assignment open	[Auto × Auto], [Auto × Human], [None × Auto]; only class where auto-gated Loop 2 needs no receiving-side human	test overfitting; scaffold bloat	DGM, AlphaEvolve kernels, SkillOpt on math
C2 observable outcome	success/failure event after one episode; noisy, confounded	RL threshold; held-out task suite; batching or agreement filter; $\tau_t > 0$	(1) noisy; (2) only if logged; (3) hardest; (4)(5)	[Auto × Auto] in simulators; [Auto × Human] in production	patch overfitting; poisoned memory; tool-output injection	Devin, Sierra, SAGE, SkillRL, PACE, Voyager
C3 judged	judge, rubric, rating, reference comparison; no ground truth	calibrated LLM-judge or reviewer; the rubric is itself a coordinate (reflexive gate)	(1) by proxy; (2) via complaints; (3) via rubric decomposition; (4); (5) proxy	[Auto × Human] naturally; [Auto × Auto] only with calibration; [Human × Human] for high stakes (registries and curation only)	Goodhart / eval fragility; rubric accretion	SkillForge, CASCADE, skills-repository review
C4 delayed, sparse	outcome at a long horizon, once per many edits	proposal pruning (semantic merge, archive selection) plus proxy milestones; no per-edit gate	(1) only at horizon; (2) rarely as a failure event; (3) temporal, open; (4); (5) replaced by novelty or taste	[Auto × Auto] through archive selection; otherwise [None × Auto] or human on both loops	drift; indistinguishable from random search	NanoResearch, CORAL, AlphaEvolve archive

C3, judged. There is no fact of the matter. An essay, a marketing page, a blog post, a design, a pitch deck, or a code review for style can be scored only by a judge: a human rater, a language-model judge, a rubric, or a comparison against references. SkillForge accepts a skill diff when its judge agrees with human engineers more than nine times in ten [36]; CASCADE consolidates memory under a human-and-agent gate [37]; Anthropic's skills repository promotes a skill when a reviewer approves the pull request. Three things change. The estimator bias ε_t becomes the judge's bias, bounded only by calibration against people, so the bounded-bias assumption of Appendix XII has to be bought rather than assumed. Residuals are captured only when someone objects, so condition 2 depends on complaint channels rather than instrumentation.

The third change defines the class: the rubric that sets the loss is itself a scaffold coordinate. Suppose a pitch-review system keeps overrating decks that claim a large market without numbers. It traces the failure to its rubric and edits "evaluate market size" into "require a numeric estimate and a source". That edit changes the gate that will judge the next edit. We call this the *reflexive gate*. Its dominant failure is Goodhart's law in the form measured for reward models, where optimizing a proxy past a point lowers the true score [38]: once the judge is the target, the scaffold learns the judge. Nor is a human judge stable; evaluators' criteria drift as they grade more outputs [39].

C4, delayed and sparse. The outcome arrives after weeks or months, once per many edits, in a world that has moved on. Research programs, product strategy, and open-ended dis-

covery are of this kind. The clearest instances are automated research pipelines such as the AI Scientist, which generates ideas, runs experiments, and writes the paper [40]. NanoResearch merges candidate skills by semantic compatibility rather than by a measured loss change [41]; CORAL promotes findings into shared memory on a heartbeat rather than on an evaluation [42]; AlphaEvolve's MAP-Elites archive keeps candidates for their diversity, not their score [30]. With no per-edit estimate to gate on, G_t degenerates to proposal pruning plus proxy milestones, and the descent argument covers only the composition of pruning and whatever gate exists at the horizon (Appendix XIII). Temporal credit assignment across many edits is open. The dominant failure is drift: the scaffold ages faster than the signal that would correct it. An optimizer in this class also cannot be told from random search by its outcomes alone; Ong et al. [43] report the same indistinguishability for harness optimizers.

The same classes on the weight side. Reinforcement learning from verifiable rewards trains weights on exact signals and lies outside the scored corpus, but pushing it into writing and other open-ended tasks has forced the construction of judged gates, the only ones a C3 scaffold loop can borrow. One group holds the gate fixed while the policy learns, as an instance-specific checklist, a pairwise judge trained on writing principles, a reference-anchored judge, or a fixed rubric of binary items [44]–[47]. A second group lets the gate move with the policy, scoring by the policy's own probability of a fixed reference or updating the rubric or judge during training [48]–[52]. Fixed versus co-evolved is exactly the reflexive-gate question. Of the co-evolving methods, only RLPR, whose reward is a probability rather than a rubric, reports agreement with human labels; none that co-evolves a rubric or judge does (Supplementary Material S6). On the scaffold side three systems report judge calibration [36], [53], [54], and none calibrates a judge that is itself being edited.

What the class predicts. If the class fixes the gate, the occupancy of the design space is no accident, and three predictions follow, which Section VII checks. First, an [Auto $\times$ Auto] system whose gate estimates the loss change per edit should run on a C1 signal or a simulated C2 environment, where outcomes are cheap to observe, and one that works on a C4 signal should prune proposals rather than estimate. Second, [Auto $\times$ Human] should hold only C2 and C3 systems, because the human on Loop 2 substitutes for the exact signal that Loop 1 lacks. Third, no scaffold system should yet calibrate a judged gate under the reflexive condition.

V. SUBSTRATES, CORPUS, AND SCORING

A. SIX SUBSTRATES

A deployment's scaffold is composed of six substrates (Table 4). They are the coordinates of θ_s, and each can be edited independently of the others. The three kinds of memory on the memory substrate follow the cognitive-architecture taxonomy of Sumers et al. [55].

The 0 to 5 rubric is the same on every substrate: 0, ephemeral; 1, persistent and local; 2, persistent with tool attachment but no feedback; 3, multi-scope or feedback-driven but without automated promotion; 4, automated feedback updates the scaffold, so one loop is closed; 5, the substrate's own top mark in Table 4. Systems that adapt several substrates at once receive an additional integrated row.

The top mark means different things on two kinds of substrate. On those that hold adaptable content (S1, S2, S5) it requires both loops. On those that hold governance machinery (S3 provenance and correction, S4 evaluation-gated tool versions, S6 evaluation-gated promotion with audit and rollback) it requires the machinery a second loop would need, not the loop itself. That is why MCP and the prompts-as-code platforms score 5 on S4 and S6 with no inner loop at all and still sit outside the two-loop cells of the design space.

Other partitions are possible, for instance separating evaluation suites from governance; the argument of Section III is invariant to the partition so long as the coordinates stay independently editable. This one recurs in practice: every integrated system in the corpus has a recognizable instance of each substrate, under different names.

B. HOW THE CORPUS WAS ASSEMBLED

The corpus holds systems that either wrap a foundation model with at least one persistent artifact updated from deployment signal or are described in public as self-improving, scaffolded, or agentic. They were found in arXiv, vendor documentation, and public repositories between January 2023 and the October 2026 cutoff. The corpus was assembled in two passes, the second adding 126 systems up to the cutoff (Supplementary Material S2). Each was scored from its own method section or documentation, never from a secondary summary.

Two filters structure the corpus (Fig. 3). The patch-local discriminator admits a system to the core corpus when it scores at least 3 on at least one substrate, that is, when some artifact persists across scope or is updated from feedback. Systems that carry the labels but fail it form a contrast corpus, kept so that the boundary can be audited rather than asserted. A stricter composite two-loop audit then asks whether Loop 1 modifies a persisted artifact from session signal and whether Loop 2 has an explicit cross-context promotion mechanism with versioning or lineage; only systems that pass both are counted as occupying the design space, and systems that fail it appear in the matrix as contrast entries, marked as such. Of the 256 systems, 204 form the core corpus and 52 the contrast corpus; 14 research systems (13 of them rows of the released table, and SkillOpt) and 3 industry exemplars (Sierra Agent OS 2.0, Cognition Devin, and Letta's Context Repositories) pass the composite audit. The text also treats ten further systems at mechanism level: SkillOpt, GEPA, Meta-Harness, CORAL, PACE, MOSS, RCL, FederatedSkill, SkillClaw, and ChronoMem. They were classified from their papers after the first pass closed and are not rows of the released table, so the corpus counts above exclude them. SkillOpt is the only one of them in the design matrix.

TABLE 4. The six scaffold substrates and the mark a system must reach to score 5.

Substrate	What persists	Score-5 mark
S1 Instructions	Project, team, or organization rules attached to a workspace	Versioned, two-loop, auto-gated promotion
S2 Skills	Procedural artifacts (code or text) loaded on demand	Skill bank self-modifies on failure, test-gated, versioned, promoted
S3 Memory	Cross-session experience (episodic, semantic, procedural)	Provenance, versioning, and correction pathways
S4 Tools	Vetted action surface and context schemas	Domain-curated bundle is the product; evaluation-gated tool versions
S5 Orchestration	Multi-role graph and routing	Population or policy updates from an outer-loop signal
S6 Governance	Versioning, promotion, audit, rollback	Dev-to-staging-to-production with evaluation thresholds, audit log, and rollback

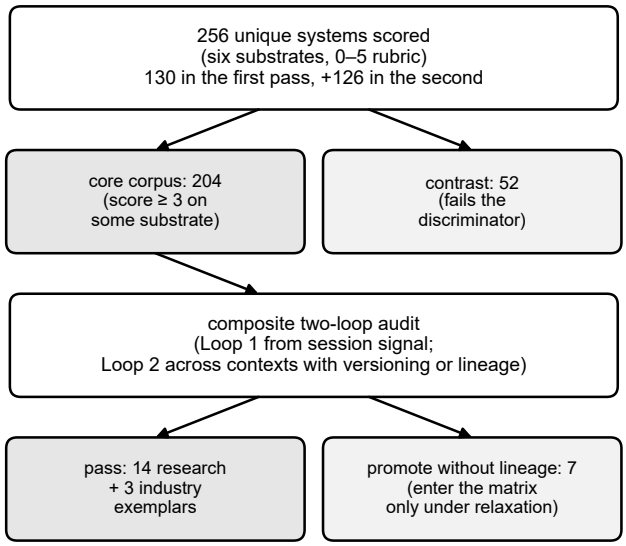


FIGURE 3. Corpus flow. Every system is scored on the substrates it adapts, with an integrated row for systems that adapt several, giving 269 substrate-level rows for 256 systems; the discriminator splits core from contrast; the composite two-loop audit selects the systems placed in the design space of Section VII.

Two kinds of system are excluded: those whose only update mechanism is weight-level fine-tuning or reinforcement learning, which are model-side and belong to the self-evolution literature this survey positions against; and those with no public evidence of deployment or release.

C. EVIDENCE AND SCORING DISCIPLINE

Every classification rests on public evidence of uneven strength and on one rater. Each system carries one or more evidence tiers (T1 to T5 in the released table: peer-reviewed publication, arXiv preprint, production claim with metrics, open-source artifact, industry blog or interview). Peer-reviewed papers and open-source artifacts weigh most in cluster-level statements; a production claim counts when it names a benchmark or integration target; a blog post or interview counts only when a higher tier corroborates it; and production claims are never pooled with peer-reviewed evidence in a cluster-level statistic. Industry systems evaluated from vendor documentation rather than a paper are flagged wherever they appear. Scoring is single-rater: one author scored every system, used half-scores only when one criterion was clearly met and another partially, and rounded ambiguous cases down. The complete score table, with a one-line justification per row, is released so that every scored classification can be re-derived (Supplementary Material S1).

The counts of the findings move under two natural relaxations of the composite audit: counting any of versioning, lineage, or rollback as Loop-2 governance, or admitting as Loop 1 an update that persists only optimizer state. The qualitative findings survive both (Supplementary Material S2).

VI. WHAT THE CORPUS SHOWS

A. MATURITY IS UNEVEN

The local loop is routine; the governed second loop has not been built. Nine in ten of the most recently added core systems close the local loop under an automatic gate, and none of them reaches the top mark on any substrate (Table 5).

On the three substrates that hold adaptable content, only two systems reach the top mark, and both reach it inside one research environment. Instruction files persist across IDEs and agents (Claude Code, Cursor, AGENTS.md [56], Windsurf, Continue.dev) and appear in a fifth to a quarter of coding-agent projects [57], [58], yet in a controlled study they raise inference cost by about a fifth without raising task success [59]. Skills are the most active research substrate: SAGE and COSPLAY close both loops [33], [60], and Loop 1 is routinely automated. SkillChain's judge-monotone edits pass a human reflection gate before they enter its skill bank: a two-stage local gate rather than a promotion loop [53]. Eleven skill systems treat retirement as a gated operation [61]–[64], and the public skill repositories are merged by hand [65]. On orchestration, automated workflow search came first, in optimizable agent graphs [66] and tree search over code-represented workflows [67]. Topologies evolve under evaluation [68]–[70], TOCOMAS alone keeps a topology change only when it satisfies a structural invariant [71], and Evo2Team promotes routing skills across teams [72].

Memory carries lineage without enforced correction. Production memory records its own history, in Letta's git-backed

TABLE 5. Substrate maturity across the 256-system corpus. Column n counts scored rows on each substrate, Mean is the average score, and Score-5 systems reach the substrate's top mark; the integrated row counts systems scored across several substrates as one composite row. Of the most recently added 126 systems, 31 are skill systems and 102 are core; 92 of the core systems close Loop 1 under an automatic gate, and none of the 126 scores 5 on any substrate.

Substrate	*n*	Mean	Score-5 systems	Survey finding
S1 Instructions	15	3.07	(none)	Rules persist across IDEs and agents; none closes both loops; ceiling 4
S2 Skills	50	3.34	SAGE, COSPLAY	Most active research substrate; Loop 1 routinely automated; production repositories human-merged
S3 Memory	50	3.06	Cuadros et al.	Lineage shipped in products; enforced correction still absent
S4 Tools	20	3.65	MCP, Harvey, Hippocratic AI	Most mature production substrate; registries govern admission, not evaluation
S5 Orchestration	37	3.08	(none)	Topologies evolve under evaluation; one structurally gated instance; no two-loop system
S6 Governance	45	3.02	Braintrust, Vellum, LangSmith Hub, AGENTS.md/AAIF	Mature prompts-as-code; registries add lineage and rollback; no inner loop
Integrated	52	3.81	NanoResearch, AutoAgent, SkillRL	High-water mark; research systems without enterprise governance

Context Repositories [73], Mem0's per-memory history [74], and Zep's bitemporal facts [75]. Research memory splits into write-gating and provenance on one side [76]–[78] and verifier-gated failure loops on the other [79]–[81]. Among the memory systems that gate writes or correct from failures, only Cuadros et al. does both, which is why it alone reaches the top mark.

Tools and governance hold the machinery of a second loop without the first, which is how they reach the top mark with no inner loop (Section V). MCP registries split into two kinds: catalogues such as the official registry, with namespaces and version records but no scanning and no private servers; and control planes, with scanning, access control, and quarantine [82]–[84]. None has evaluation-gated promotion. The governance registries bring lineage and rollback [85]–[87], and none of them has an inner loop.

No system matures all six substrates at once. Research systems learn fast and lack enterprise governance; production systems govern well and learn slowly; open standards solve artifact portability but not adaptation. This pattern is not a quirk of the corpus: it tracks which constraints each kind of system was built to satisfy.

B. FIVE ABSENCES, READ BY SIGNAL CLASS

Five pieces of the loop are missing or rare across the core corpus, and each sits in a particular feedback-signal class, or in all of them.

- **Failure-triggered memory is rare in production** (C2). Deployed memory systems record preferences and successful facts, not residuals. The one field report of stores written from dead-ends to prevent recurrence describes a single, self-reported deployment [88].
- **Cross-customer promotion is nearly absent** (all classes). ShareMem, EvolveNet, XRepoSkill, and RASO promote across users, clients, or repositories inside one operator [89]–[92]. No system promotes across customers or organizations with evidence.
- **Memory is versioned, but correction is not enforced** (C2, C3). Git-backed and bitemporal memory ship in products. Five production backends still return facts that were revoked [93].
- **Scaffold-level curriculum is unspecified in nearly every system** (C2). Which sessions inform the next edit is left to the order in which they arrive. Task-CoEvolve's variance-weighted task sampling and AutoSaddler's mini-batch failure signals are the exceptions [94], [95].
- **Receiving-side evaluation at promotion is rare** (C1, C2). ShareMem gives evaluation tasks to a different user, XRepoSkill verifies in at least two repositories, Evo2Team confirms at the target team, and VACE evaluates the candidate with the receiving model held fixed [96]. Even Evo2Team's confirmation lets nearly a fifth of its transfers fail on new graphs. EvolveNet has no such check, and the federated optimization framework that proposes a receiving-client gate has not implemented it [97].

C. FOUR CLUSTERS

Four clusters recur across the core corpus (Table 6). They differ in which loop is automated and in how far each sits on the gradient from patchwork to descent.

Research full-auto systems show that the architecture works when both gates are algorithmic [29], [30], [41], [98]–[101]. AlphaEvolve is the clearest case: it pairs an evaluator with an evolutionary archive, as FunSearch did before it, and reports production gains at Google. EvolveNet, Evo2Team, and XRepoSkill extend the loop from a single deployment to a federation of deployments. No member has access control or audited promotion.

Production hybrid systems show that the production-viable pairing is an algorithmic gate on Loop 1 with a reviewer on Loop 2. SkillForge illustrates it most plainly: a judge that agrees with people more than nine times in ten passes

TABLE 6. Four clusters of the core corpus.

Cluster	Representative systems	Loop 1 / Loop 2	What the cluster shows
Research full-auto	NanoResearch, SkillOpt, SkillRL, AlphaEvolve, DGM, ADAS, Voyager, EvolveNet, Evo2Team, XRepoSkill	Auto / Auto	Both loops can run under algorithmic gates and benchmark gains compound; enterprise governance is uniformly absent
Production hybrid	SkillForge, CASCADE, Sierra Agent OS 2.0, Cognition Devin, Spotify's recommendation loop [28]	Auto / Human	An algorithmic Loop-1 gate with a reviewer on Loop 2 is the production-viable cell; Spotify adds online A/B gating
Governance-first	Braintrust, Vellum, LangSmith Hub, EDDOps, Agentic Configuration Management, Governed Capability Evolution	None / Auto or Human	Evaluation-gated promotion, lineage, and rollback exist; candidate generation is manual and no inner loop fires from session signal
Open standards and registries	AGENTS.md, MCP, Anthropic Agent Skills, Cursor Rules, the MCP control planes	Mixed, or Human / Human	Cross-vendor convergence on artifact format and tool attachment; registries govern admission and access but do not evaluate

each skill diff, and support engineers decide whether it ships [36]. Sierra routes agent-generated insights through human-authored expert answers [102], Devin's merged pull requests are its promotion step [32], and Spotify launches automatically optimized prompts and plans only behind an online A/B test. In every member the human on Loop 2 does the work that the missing exact signal cannot.

Governance-first systems automate Loop 2 alone. The prompts-as-code platforms [103]–[105] and the governance registries [85]–[87] keep immutable versions, evaluation thresholds, and rollback, but they have no Loop 1: engineers author the candidates, and session feedback never triggers one. This is MLOps for scaffolds, distinct from MLOps for weights.

Open standards and registries converge across vendors on format and attachment. Claude Agent Skills, Cursor Rules, GitHub Copilot instructions, Windsurf, Continue.dev, Zed, and Replit all keep a git-tracked markdown artifact attached to a workspace with an MCP-style tool layer [58], [106], a convergence that Section VIII reads as evidence that the architecture is real. What the cluster lacks is adaptation: its registries govern admission and access but do not evaluate, and in the repositories that run autonomous loops, the runtime state of those loops sits outside version control [107].

VII. THE TWO-LOOP DESIGN SPACE

Indexing systems by the gate on each loop gives a 3×3 matrix, and its occupancy is lopsided (Fig. 4).

[Auto $\times$ Auto] holds the research full-auto systems, machine-evaluated on both loops, and Letta's Context Repositories, which push shared memory across agents automatically, with git lineage and no evaluation gate. That push is unconditional, an algorithmic promotion with no evaluation threshold, so it sits in the Auto column of Loop 2; the composite audit admits it on lineage, not on a gate. [Auto $\times$ Human], the production-viable cell, holds SkillForge, CASCADE, Sierra Agent OS 2.0, and Cognition Devin: Loop 1 fires at machine speed under an algorithmic criterion, and Loop 2 waits for a person. [None $\times$ Auto] holds the governance-first platforms, which gate promotion on evaluation but generate

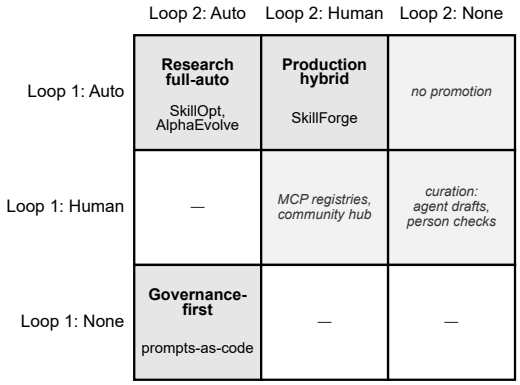


FIGURE 4. The two-loop design space. Rows give the Loop 1 gate and columns the Loop 2 gate, so a cell [Loop 1 $\times$ Loop 2] such as [Auto $\times$ Human] pairs an algorithmic local gate with a human promotion gate. Two cells hold every system that passes the composite audit; a third holds the governance-first platforms, which have the gated second loop but no inner loop; the human-authored cells are occupied only by registries and curation workflows that have no session-signal loop. No populated system adds governed cross-tenant promotion.

no candidates from failures; they pass only the Loop-2 half of the audit and enter the matrix as contrast entries (Supplementary Material S4).

Seven systems promote across users, clients, teams, or repositories without versioning or lineage. Six of them would enter the matrix if any of versioning, lineage, or rollback counted as Loop-2 governance, landing in [Auto $\times$ Auto] except Spotify's production loop, which lands in [Auto $\times$ Human]. The seventh, Future AGI's prompt lifecycle, rests on a vendor blog alone (tier T5), which the tier rule does not count, and is not admitted under any rule. None opens a new cell (Supplementary Material S2).

The other cells, where occupied at all, hold only systems that fail the composite audit. [Human $\times$ Auto] is empty: nobody authors candidates by hand and promotes them automatically, because automating the promotion of hand-written candidates buys little. [Human $\times$ Human] holds MCP reg-

istries and a community hub [82]–[84], [108], where a person registers an artifact and a person approves its use. The field study of five public skill repositories finds the same shape in practice, with every substantive edit human-merged and one deprecation in 254 edits [65]. [Human × None] is the curation cell. An agent drafts, a person fact-checks, and the result is frozen [109]. These are single-loop by construction. The Loop 1 label in these cells, as in [None × Auto], records who supplies each candidate: a person who registers an artifact for a specific use (Human) or an engineer who authors a version at a desk for no particular failure (None); the distinction is one of convention, and the supplementary matrix records it (Supplementary Material S4).

Read by signal class, the occupancy bears out the first two predictions of Section IV. Every [Auto × Auto] system whose gate estimates the loss change per edit runs on an exact signal or a simulated outcome, and those that work on delayed signals prune proposals instead. [Auto × Human] holds only systems whose signal is an observed outcome or a judgment; no exact-signal system sits there. The third is the scaffold-side finding that section already records, that three systems report judge calibration and none calibrates a judge under edit, and no system in the matrix changes it.

Evidence is batched before acceptance, whereas earlier systems accepted an edit on one evaluation and absorbed estimator noise through the per-edit margin alone. A paired per-item test commits an edit only when its wins are reliable [110]; a merge is accepted only when it fixes at least as many held-out items as it breaks [90]; three heterogeneous critics must all pass [111]; mini-batch failure signals feed validation-based selection [95]. Requiring agreement among independent proposals and averaging over them are different operations, and they reduce error at different rates (Appendix XIII).

Multi-layer hierarchies remain rare. AlphaEvolve is the only system that combines a cascade evaluator with a diversity archive; the other systems that keep archives use them for lineage, not for staged evaluation.

VIII. CONVERGENT RESEARCH FAMILIES

The loop is not a prescription the field has yet to adopt. Five research families, starting from different substrates and under different names, have arrived independently at parts of it, which we read as evidence that the loop is a shared attractor rather than one group's local optimum. The vendors' convergence on one artifact format and tool attachment (Section VI) is independent evidence of the same kind. Convergence does not show that the design is optimal, only that the same constraints keep producing the same shape.

The clearest single instance is SkillOpt [31]. It treats one markdown skill document as the trainable external state of a frozen agent and edits it with explicit optimizer controls: rollout and reflection batches, add/delete/replace edits, a bounded textual learning rate with cosine decay, a held-out selection gate that accepts only a strictly better score, a rejected-edit buffer kept as negative feedback, and an epoch-wise slow update. Across six benchmarks, seven target models, and three harnesses it is best or tied on every cell its authors evaluate. It also reports transfer across models, harnesses, and benchmarks, which reads as an empirical measurement of evidence radius rather than as a governance mechanism. Its own follow-up, SkillOpt-Lite [112], removes the buffer, the slow update, and the schedule, keeps the held-out gate and a consensus filter across rollouts, and matches or beats it. The gate and a consensus form of credit assignment are load-bearing; the rest are stabilizers.

The five families instantiate complementary parts of the loop (Table 7), and each was pushed into its part by the constraint it started from. Reflective evolution started from trajectories and proposes edits by reflecting over them, with a two-timescale argument for keeping those edits outside the weights [4], [113]. Reflective context learning, the most direct independent formalization, started from the observation that credit assignment, overfitting, forgetting, and high-variance signals persist whether the learned object lies in parameter space or context space. Its generalization penalty is the complexity term Ω of the gate [114]. Harness optimization started from orchestration rather than skills and names credit assignment as its bottleneck; it is the largest family in the corpus, with roughly two hundred papers citing its founding entries and a dozen scored here [70], [115]–[120]. The lifecycle family started from artifact pools that bloat and found hygiene to be a precondition for descent [121]: the complexity penalty becomes a constraint frontier once platform limits bind [122], each outcome is attributed to skill, exploration, or environment before an update is admitted [123], and eleven systems make retirement a gated operation [61]–[64]. The transfer family started from how far an edit travels and instruments Loop 2, from measuring evidence radius one task at a time to federating skill patches across clients or deployments [35], [42], [90], [124]–[126].

Read against the loop, the families support four conclusions.

1. The scaffold-versus-weights decomposition is independently confirmed. GEPA, Meta-Harness, MOSS, RCL, PACE, and the harness-optimization papers all begin from the observation that weights and the external artifact layer are distinct adaptation surfaces with different cost, reversibility, and scope; a system that updates both keeps them as separate levers [127].
2. Credit assignment is the central engineering bottleneck. AHE, CANTANTE, Meta-Harness, HARNESSEVO, SkillsVote, and the harness-optimizer evaluation of Ong et al. [43] all name the same step as what separates descent from trial-and-error: attributing a system-level failure to one scaffold coordinate. That step remains unsolved in the general multi-substrate case.
3. Lifecycle hygiene is a precondition for descent, not an alternative to it. A dirty artifact pool starves the gate of signal (Table 9), and gated retirement is the field's answer. The one negative result cuts deeper: once a contaminated skill has descendants, rolling it back recovers little, so gating must happen before commit [111].

TABLE 7. Five research families converging on artifact-layer descent, with the part of the loop each instantiates.

Family	Representative systems	What it instantiates	What it leaves open
Reflective evolution	GEPA; Tiwari et al.	proposal by reflection over trajectory evidence; two-timescale motivation	persistent artifact; bounded step; held-out gate
Reflective context	RCL	the learned object in context space; generalization penalty as Ω	cross-deployment Loop 2
Harness optimization	Meta-Harness; AHE; MOSS; CANTANTE; HARNESSEVO; HarnessForge; Growing Harness	credit assignment over orchestration; source-level edits; slot attribution	governed Loop 2; access control; rollback
Lifecycle and multi-objective	Ratchet; MOCHA; SkillsVote; Library Drift; SkillSmith; ASSAY	hygiene as precondition; constraint frontier as the shape of Ω; gated retirement	gradient-analogue closure; retirement of a promoted artifact's descendants
Transfer and federation	ICT; Combee; CORAL; PACE; FederatedSkill; EvolveNet; Evo2Team; XRepoSkill	empirical evidence radius; cross-client aggregation; receiving-side confirmation	cross-organization gate, lineage, rollback

4. No surveyed system combines automated local adaptation with governed cross-organization promotion. FederatedSkill and EvolveNet each narrow the gap on different properties, and neither meets the four that Section IX sets out.

IX. THE MISSING ARCHITECTURE: GOVERNED CROSS-TENANT SCAFFOLD OPTIMIZATION

Four properties define an architecture that no surveyed system instantiates:

- (a) **auto-gated Loop 1**: the commit criterion is an algorithmic threshold;
- (b) **cross-organization promotion**: Loop 2 routes artifacts between organizations (we use *cross-tenant* and *cross-organizational* interchangeably);
- (c) **versioned lineage with access control**: promoted artifacts carry versions, audit trails, and role-based access policies;
- (d) **rollback**: a bad promotion can be reverted without redeploy.

The closest approaches split these properties among them, and no row of Table 8 is full on promotion across organizations.

Row by row, the partition is plain: the systems with a local loop carry the automatic gate and part of the promotion, the registries carry lineage and rollback without an inner loop, and the control planes carry access control alone. Rollback appears inside one federated round and inside single deployments, as edit-level reversal [62], [128] and as an admission test that rejects edits which cannot be undone [129]. No system rolls back a promotion that has crossed an organization boundary, because none makes such a promotion (Supplementary Material S4).

In machine-learning terms, what is missing is a cross-tenant aggregator with an automated evaluation gate at the scaffold layer. It would carry five parts: per-tenant artifact pools, each tagged with provenance; an aggregation step across tenants that respects tenant isolation; a held-out evaluation at the receiving tenant that blocks a promotion whenever it does not lower that tenant's patch loss; versioned lineage with access control; and a rollback envelope on every promoted artifact. The federated-learning analogy [130]–[132] is useful only in the limit where artifacts are mergeable update representations; federated prompt tuning, in which participants learn a shared prompt rather than shared weights [133], is the weight-side precedent closest to that limit. In surveyed practice Loop 2 is selection and merge, and the nearest systems aggregate program edits with a language model, not by averaging.

Whether the architecture *should* be built depends on the feedback-signal class, not on the application domain (Section IV). Where the signal is exact, automatically gated cross-tenant promotion of a test-validated skill would be low-risk, and receiving-side evaluation is cheap to add. Where it is an observable outcome, the sending side's outcome does not transfer, so promotion should be automated only behind receiving-side evaluation, the check that would catch a scaffold over-fitted to one deployment before it propagates to another. Even that check is necessary rather than sufficient: some of Evo2Team's transfers still fail on new graphs after target-side confirmation [72]. Where it is judged, a human gate on Loop 2 is a design feature, and governance means lineage and rollback around a human decision. Where it is delayed, property (a) cannot be met, and what remains is lineage and rollback without an automatic gate. Safety-critical domains, the usual case for human gates, are judged or delayed domains with high stakes. Naming the four properties turns a vague gap into an audit criterion that any future system can be measured against; naming the signal class says which of the four a given deployment should expect to meet.

X. THE FAILURE SURFACE

Patchwork scaffold maintenance introduces failure modes that weight-only systems do not have; they are the scaffold-layer entries in the technical-debt ledger that machine-learning systems have carried for a decade [134]. Each is also a missing piece of the optimizer, and each dominates in a particular feedback-signal class (Table 9). The entries are documented attacks and degradations, not a new threat model.

TABLE 8. The closest approaches to governed cross-organization scaffold optimization, scored against the four properties.

System	(a) auto-gated Loop 1	(b) cross-organization promotion	(c) versioned lineage with access control	(d) rollback
FederatedSkill [126]	full	partial: privacy-preserving aggregation of skill patches across heterogeneous clients that sit under one operator	partial: auditable per-modification decision log	none
EvolveNet [90]	full	partial: held-out merge gate at the aggregator, but clients are benchmark partitions, not organizations	none: no lineage, no access control	full: rollback to the previous shared harness after a failed round
SkillForge [36]	full	partial: cross-team within one enterprise	partial	not documented
Governance-first registries [85]–[87]	none: no inner loop	none	partial: lineage without access control	full, with evidence
MCP control planes [82]	none: no inner loop	none	partial: access control without evaluation-gated promotion	not reported

TABLE 9. The failure surface of patchwork scaffold maintenance, the measured evidence, the architectural response, and the feedback-signal class in which each failure dominates.

Failure	What has been measured	Architectural response	Dominant class
Scaffold bloat	Instruction-following accuracy falls and latency grows sharply as instruction count scales [135]; LLM-authored skills add nothing over no skill until lifecycle hygiene is applied [121]	Complexity penalty Ω in the gate; pruning and expiry as first-class gated operations	C1, C3
Poisoned memory	High attack success against standard retrieval-augmented generation [136]; backdoors planted in long-term memory or knowledge bases [137]; memory control-flow attacks across major agent frameworks [138]	Provenance on every entry; write gating; a rollback envelope [139], [140]	C2
Injection via tools and MCP	A tool's output or a connected server can smuggle instructions into context [141], [142]; one approved MCP configuration could later be modified without re-approval (CVE-2025-54136) [143]	Narrow tool permissions; a privilege hierarchy over instructions [144]; tool bindings pass the same promotion gate as any other coordinate	C2
Evaluation fragility	Optimizing a proxy reward past a point lowers the true score [38]; access asymmetries alone produce large relative gains that reflect exposure to the test [145]; harness optimizers cannot be told from trial-and-error by downstream score alone [43]	Held-out evaluations scoped within evidence radius, weighted toward recurring modes, re-read as the target drifts, and checked against an update's attribution	C3 (primary), C1
Coordination failures	Fourteen failure modes in multi-agent systems, dominated by role confusion and context loss [146]	Orchestration topology treated as a scaffold coordinate under the same gate and rollback discipline	C2
Patch overfitting	A scaffold tuned to one deployment over-fits it by construction (Supplementary Material S3)	Evidence-radius discipline at promotion; receiving-side held-out evaluation	C2
Plasticity loss	Context rot (long-context degradation [147]), instruction conflict, memory pollution, tool-version drift	Expiry and rollback as explicit loop operations rather than retrospective hygiene	C4

Bloat is a signal-quality problem as well as a latency problem. Ratchet reports that LLM-authored skills gain nothing over a no-skill baseline while human-curated ones show a double-digit gain, and it attributes the gap to lifecycle management [121]. In the vocabulary of Section III, that means residual capture and mode discovery cannot work on a messy artifact pool.

Evaluation fragility compounds because the evaluation suite is itself a patch-local artifact rather than a fixed instrument. It is optimized against: once a metric becomes the target, it is gamed. It is read non-independently, because the gate scores path-dependent agent trajectories whose readings do not compose like ordinary test-set statistics. It goes stale, because the suite adapts on the same cadence as the scaffold it gates. And because failures concentrate in a finite catalogue of recurring modes, a suite that samples tasks at random under-weights exactly the families that drive the failure rate. The response in the table follows from those four facts: held-out and scoped, mode-weighted, re-read, and checked against attribution rather than only against score.

The failure surface is real, quantified in the cited measurements, and largely unmitigated in production. Every row

names a piece of the optimizer that current systems do not yet have.

XI. OPEN PROBLEMS AND CONCLUSION

Five open problems follow from the survey, one per feedback-signal class and one for the architecture.

1. **Exact signals: credit assignment from a failing check to a scaffold coordinate.** A test tells the loop that an output failed, not which instruction, skill, or tool binding caused it. Slot attribution [119], intervention-based attribution [69], and contrastive multi-agent attribution [118] each solve one substrate's case; a general method is the bottleneck the harness-optimization family names.
2. **Observable outcomes: receiving-side evaluation before promotion.** The outcome that justified an edit on the sending side does not transfer, and even where a promoted edit is checked on arrival, some transfers still fail (Section VI). The gate a federated scaffold optimizer needs, a held-out test at the receiving tenant, has been proposed but not built [97].
3. **Judged signals: calibrating a gate that is itself being edited.** When the rubric is a scaffold coordinate, each accepted edit to it moves the instrument that judges the next edit. No method that co-evolves a rubric or judge reports agreement with human judgment, and no scaffold system calibrates a judge under edit (Section IV); the nearest measurement is on the human side, where evaluators' criteria drift as they grade [39].
4. **Delayed signals: telling descent from search.** Where the outcome arrives after many edits, an optimizer's gains cannot be distinguished from trial-and-error by outcome alone [43]; evaluating harness optimizers against their attribution claims rather than their scores is the open method.
5. **The governed cross-organization loop.** The architecture of Section IX, properties (a) through (d), remains unbuilt; the receiving-side gate of problem 2 is the piece that would make cross-organization promotion safe in the two classes where it should be automatic.

The field has built an external adaptation layer for language models but not the optimizer that should govern it. The two earlier papers of this series showed why the layer belongs where it is: errors concentrate in a few key decisions; inside a bounded deployment, those decisions fail in a small and recurring catalogue of modes; and such a catalogue can be fixed one mode at a time in a persistent, inspectable, reversible artifact outside the weights. Gated and scoped, that artifact layer is the architecture the error structure calls for, not a workaround.

The survey shows how far practice has come toward that optimizer, and the distance is uneven. The local loop closes automatically in most research systems and in a growing number of production ones; evidence is batched before acceptance; retirement and expiry are first-class operations; production memory carries lineage. What has not moved is the second loop. No surveyed system promotes a validated edit across an organization boundary under an automatic gate with lineage, access control, and rollback. The signal-class analysis says this is not one gap but four, each with its own reachable answer. The frontier model generalizes. The local scaffold specializes. Reliability comes from governing that specialization. Because the instruments that judge each edit are themselves part of the moving scaffold, the same evidence-radius and lifecycle discipline the optimizer needs must extend to the gates that judge it.

XII. APPENDIX: FORMAL STATEMENT OF ARTIFACT-LAYER DESCENT

This appendix states the setup behind Section III, a one-step descent inequality under a noisy gate, an informal convergence proposition, the optimization-analogy correspondence, and four worked cases. No new theorem is claimed; the proposition is the ordinary target of local stochastic optimization under noisy estimators, restated for a discrete, versioned parameter.

A. SETUP

Let the frontier model have fixed parameters θ_M and let the scaffold be a structured object $\theta_s \in \Theta_s = \Theta_1 \times \cdots \times \Theta_6$ over the six substrate coordinates of Section V; Θ_s is discrete, symbolic, and versioned. For a deployment D, $L_D(\theta_s) = \mathbb{E}_{x\sim D}[\ell(\pi(\theta_M, \theta_s, x))]$. A candidate edit $z_t \sim Q_t(z \mid H_t, \theta_s^t)$ is drawn from a proposal process conditioned on the accumulated history H_t of failures, corrections, rejected outputs, evaluation regressions, and prior accepted edits. Q_t is not uniform: for a mode m that recurs with frequency f_m and carries per-occurrence loss s_m, its expected contribution to the failure rate is $f_m s_m$, so high-frequency or high-severity modes are proposed first. The finite directional difference is $\Delta_z L_D(\theta_s) = L_D(\theta_s \oplus z) - L_D(\theta_s)$; the gate sees only a noisy estimate $\widehat{\Delta}_z L_D$. The update is

$$\theta_s^{t+1} = \theta_s^t \oplus z_t \text{ if } G_t(z_t) = 1, \quad \theta_s^{t+1} = \theta_s^t \text{ otherwise.}$$

B. DESCENT INEQUALITY

Assume (A1) positive descent bias on accepted edits, $\mathbb{E}[\widehat{\Delta}_{z_t} L_D \mid G_t = 1, \theta_s^t] \le -\gamma_t - \tau_t$ for some $\gamma_t > 0$, and (A2) bounded estimator bias on accepted edits, $|\mathbb{E}[\Delta_{z_t} L_D - \widehat{\Delta}_{z_t} L_D \mid G_t = 1, \theta_s^t]| \le \varepsilon_t$. With $p_t = \Pr(G_t = 1 \mid \theta_s^t)$,

$$\mathbb{E}[L_D(\theta_s^{t+1}) \mid \theta_s^t] \le L_D(\theta_s^t) - p_t(\gamma_t + \tau_t) + p_t\,\varepsilon_t.$$

Per-step expected improvement is the gate's margin minus its estimator bias, weighted by acceptance probability. In the feedback-signal classes of Section IV, ε_t is near zero for an exact check (C1), bounded by batching for an observable outcome (C2), equal to the judge's bias and bounded only by calibration for a judged signal (C3), and unavailable per edit for a delayed signal (C4).

C. CONVERGENCE (INFORMAL)

Add (A3) proposal coverage, that at every non-gate-stable scaffold Q_t places probability at least $q > 0$ on an admissible edit with $\mathbb{E}[\Delta_z L_D] \le -\delta$; (A4) effective compactness

TABLE 10. Optimization concepts and their scaffold analogues.

Optimization concept	Scaffold analogue
Parameter vector	Instructions, skills, memories, tools, routing graphs, evaluation gates, governance rules
Loss	Failure, user correction, evaluation regression, retrieval miss, tool error, schema violation
Gradient estimate	Credit assignment identifying the coordinate responsible for a failure
Step	Accepted scaffold edit; step size is edit magnitude under $\oplus$
Aggregated update	Several recurring failures combined before an update, by averaging or by agreement across k proposals
Update radius	Promotion scope: which loss is descended (USER, PROJECT, STACK, TENANT, CORE)
Validation loss	Held-out patch evaluation before merge
Regularization	Complexity penalty $\Omega(z)$, bloat control, cross-patch regression checks
Overfitting	Scaffold improves on this deployment and degrades elsewhere
Rollback	Reverting a harmful accepted update

of the reachable orbit; and (A5) persistent acceptance with summable bias, $\sum_t p_t(\gamma_t + \tau_t) = \infty$ and $\sum_t p_t \varepsilon_t < \infty$; and let $0 \le L_D \le B$. Then $L_D(\theta_s^t)$ is a non-negative supermartingale up to summable noise, it converges almost surely, and the limit set of $\{\theta_s^t\}$ is gate-stable: no admissible edit clears the gate with negative expected directional loss in the limit. This is convergence of the loss and stability of the gate decision, not convergence to an optimum; a globally suboptimal gate-stable scaffold is consistent with the result, because Q_t may never propose the improving edit or $\widehat{\Delta}$ may be biased to reject it. In a non-stationary deployment ($D = D_t$) the same mechanism tracks a moving optimum rather than converging once, which is why recency weighting, pruning, expiry, and rollback are the scaffold-level analogues of continual-learning machinery rather than optional hygiene.

D. CORRESPONDENCE

Table 10 pairs each optimization concept with its scaffold analogue. Credit assignment here is attribution, not back-propagation. Scaffold systems do not differentiate through instruction files; they trace an output error to an upstream coordinate, manually or with a judge or verifier. The descent argument needs only that this attribution succeeds with non-zero probability per failure, which is the credit-assignment condition of Section III.

E. WORKED CASES

Retrieval. Failures recur because retrieved context omits the decisive clause. Attribution traces the error to the retrieval coordinate: chunking, query rewriting, metadata filter, embedding model, reranker, or context-packing rule. The candidate edit tightens chunk boundaries or adds a metadata constraint, and the gate checks that recall and answer accuracy improve on the patch evaluation without degrading adjacent cases. Signal class C1 or C2.

Pitch review. The system repeatedly overrates decks that claim a large market without numbers. The failure cluster points at an under-specified rubric coordinate. The edit changes ''evaluate market size'' into ''require a numeric estimate and a source; if absent, cap the score''. The gate runs the revised rubric against held-out decks with human scores. The edit changed the gate that will judge the next edit: the reflexive gate of Section IV, class C3.

Tool use. Calls fail because optional parameters are under-specified. The responsible coordinate may be the tool schema, the call policy, or the clarification rule. The edit adds required fields or a pre-call uncertainty check; the gate checks that tool-call failures fall without raising refusals or latency. Class C2.

Memory. A stale customer preference is repeatedly retrieved and causes bad decisions. The responsible coordinate is a memory entry together with its provenance, priority, and expiry policy. The edit corrects, expires, or version-tags the entry; the gate checks that future behavior improves without erasing useful history. Class C2, with the correction itself observable only when a later outcome reveals it.

XIII. APPENDIX: GATE FAMILIES AND FEEDBACK-SIGNAL CLASSES

Section III collapses every production gate into one per-edit test G_t on an estimate of the loss change. Surveyed systems instantiate that test in several distinct ways, and some of them are better read as proposal pruning that precedes the per-edit gate than as estimators of $\widehat{\Delta}L_D$. Table 11 lists the families, the slot each occupies, representative systems, and the feedback-signal class (Section IV) each family serves.

The first five families populate G_t directly and inherit the descent guarantee of Appendix XII under its assumptions. Semantic merge and archive selection are not estimators of $\widehat{\Delta}L_D$; they constrain the proposal distribution, and the descent argument applies to the composition of pruning and gate. Multi-sample agreement is listed because it is the correct analogue of ''wait for k pieces of evidence before accepting an edit'', a different operation from mini-batch averaging and one that should not be conflated with it. Read down the signal-class column. The only families available to a judged (C3) deployment are the two whose estimator is a judge, and the only families available to a delayed (C4) deployment are the two that do not estimate the loss change at all. That is the structural content of Section IV.

TABLE 11. Gate families, their slot in the formalism, and the feedback-signal class each serves.

Gate family	What it estimates	Slot in Section III	Signal class	Representative systems
Surrogate verifier	Pass or fail from tests, a symbolic checker, or an exact match	Per-edit gate G_t	C1	DGM (sandboxed code), SkillOpt (held-out exact match), Voyager (game-state check: C2)
Evaluation cascade	$\widehat{\Delta}L_D$ on a held-out task suite, cheap-to-expensive evaluators	Per-edit gate G_t	C1 when the suite is exact; C2 when it is a task suite	AlphaEvolve, Braintrust, Vellum, LangSmith Hub
RL threshold	Reward improvement against a deployment threshold	Per-edit gate G_t	C2	SkillRL, SAGE, EvolveR, PACE
LLM judge with calibration	Pairwise or rubric score, calibrated against human agreement	Per-edit gate G_t	C3	SkillForge (Loop 1), CASCADE (consolidation gate)
Reviewer judgment	Human credit assignment by pull-request review or expert sign-off	Per-edit gate G_t, human-instantiated	C3 (and C2 when the reviewer has outcome data)	SkillForge (Loop 2), Sierra Agent OS 2.0, Anthropic skills repository
Semantic merge	Whether a candidate duplicates or conflicts with existing artifacts	Proposal pruning before G_t	C4 (also a pre-filter in C2, C3)	NanoResearch orchestrator, FederatedSkill server agent
Archive selection	Whether a candidate fills an unfilled cell in a behavior-diversity archive	Proposal pruning before G_t	C4	AlphaEvolve (MAP-Elites), ADAS
Multi-sample agreement	Whether k independent proposals or evaluations agree; false-accept rate decays as p^k rather than variance as $1/k$	Aggregation filter on the estimate	C2, C3 noise absorption	VaG (three heterogeneous critics must all pass); SkillOpt-Lite (consensus filter across rollouts)

ACKNOWLEDGMENT

The authors used an AI assistant (Claude, Anthropic) to help search the literature, check citations, and edit prose; all claims, scores, and classifications were reviewed and approved by the authors. Earlier versions of this argument appeared as an arXiv preprint and as a non-archival workshop submission; this article extends them with the feedback-signal classification of Section IV and a corpus scored through October 2026.

REFERENCES

[1] M. Zaharia, O. Khattab, L. Chen, J. Q. Davis, H. Miller, C. Potts, J. Zou, M. Carbin, J. Frankle, N. Rao, and A. Ghodsi, "The shift from models to compound AI systems," Berkeley Artificial Intelligence Research (BAIR) Blog, https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/, 2024.

[2] R. Kirk, I. Mediratta, C. Nalmpantis *et al.*, "Understanding the effects of RLHF on LLM generalisation and diversity," in *ICLR 2024*, 2024.

[3] V. Padmakumar and H. He, "Does writing with language models reduce content diversity?" *arXiv preprint*, 2023.

[4] R. Tiwari, K. Sareen, L. A. Agrawal, J. E. Gonzalez, M. Zaharia, K. Keutzer, I. S. Dhillon, R. Agarwal, and D. Khatri, "Learning, fast and slow: Towards LLMs that adapt continually," *arXiv preprint*, 2026.

[5] S. Dohare *et al.*, "Loss of plasticity in deep continual learning," *Nature*, vol. 632, no. 8026, pp. 768–774, 2024.

[6] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," *arXiv preprint arXiv:2106.09685*, 2021.

[7] M. L. Arbuzov, S. Bei, Z. Dong, D. Kalaev, and A. A. Shvets, "Beyond exponential decay: Rethinking error accumulation in large language models," *arXiv preprint*, 2025.

[8] M. L. Arbuzov, L. Mosbacker, S. Bei, Z. Dong, D. Kalaev, and A. Shvets, "The architecture of errors: From universal impossibility to patch-local LLM reliability," *arXiv preprint arXiv:2605.30628*, 2026.

[9] H.-a. Gao, J. Geng, W. Hua, M. Hu, X. Juan *et al.*, "A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence," *Transactions on Machine Learning Research*, 2026, arXiv:2507.21046v4, 16 Jan 2026.

[10] J. Fang, Y. Peng, X. Zhang, Y. Wang, X. Yi, G. Zhang, Y. Xu, B. Wu, S. Liu, Z. Li, Z. Ren, N. Aletras, X. Wang, H. Zhou, and Z. Meng, "A comprehensive survey of self-evolving AI agents: A new paradigm bridging foundation models and lifelong agentic systems," *arXiv preprint arXiv:2508.07407*, Aug. 2025.

[11] H. Zhou, H. Hu, T. Luo, Y. Shang, C. Fang, Z. Chen, L. Xiao, and Q. Zhang, "Self-evolving coding agents," *arXiv preprint arXiv:2608.03392*, Aug. 2026.

[12] T. Ding, A. Nannapaneni, B. Liu, and L. Zhang, "Always-on agents: A survey of persistent memory, state, and governance in LLM agents," *arXiv preprint arXiv:2606.30306*, 2026.

[13] Z. Lin, X. Hao, R. Fu, S. Cui, K. Chen, C. Li, Z. Li, and F. Xiong, "A survey on long-term memory security in LLM agents: Attacks, defenses, and governance across the memory lifecycle," in *Proceedings of EMNLP 2026*, 2026, arXiv:2604.16548v3, 22 Sep 2026.

[14] J. Guo, Z. Hao, C. Wang, C. Fan, T. Luo, H. Li, Y. Gao, H. Mei, J. Peng, R. Xu, M. Dong, H. Wu, M. Zheng, K. Han, S. Wang, C. Xu, and Y. Wang, "From question answering to task completion: A survey on agent system and harness design," *arXiv preprint arXiv:2606.20683*, Jun. 2026.

[15] B. Xia, L. Zhu, Z. Xing, Q. Lu, D. Sejdinovic, and X. Xu, "Harnessing agent skills: Architectural patterns and a reference architecture for skill-mediated LLM agents," *arXiv preprint arXiv:2606.20631*, May 2026.

[16] K. Ding, Y. Zhou, C. Jin, F. Tong, M. Zhou, and D. N. Metaxas, "Agent skill evaluation and evolution: Frameworks and benchmarks," *arXiv preprint arXiv:2606.11435*, Jun. 2026.

[17] L. Mei, J. Yao, Y. Ge, Y. Wang, B. Bi, Y. Cai, J. Liu, M. Li, Z.-Z. Li, D. Zhang, C. Zhou, J. Mao, T. Xia, J. Guo, and S. Liu, "A survey of context engineering for large language models," *arXiv preprint arXiv:2507.13334*, Jul. 2025.

[18] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2309.03409.

[19] Q. Guo, R. Wang, J. Guo, B. Li, K. Song, X. Tan, G. Liu, J. Bian, and Y. Yang, "EvoPrompt: Connecting LLMs with evolutionary algorithms yields powerful prompt optimizers," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2309.08532.

[20] C. Fernando, D. Banarse, H. Michalewski, S. Osindero, and T. Rock-

täschel, "Promptbreeder: Self-referential self-improvement via prompt evolution," *arXiv preprint arXiv:2309.16797*, 2023.

[21] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "DSPy: Compiling declarative language model calls into self-improving pipelines," *arXiv preprint arXiv:2310.03714*, 2023.

[22] M. Yuksekgonul, F. Bianchi, J. Boen, S. Liu, Z. Huang, C. Guestrin, and J. Zou, "TextGrad: Automatic "differentiation" via text," *arXiv preprint*, 2024.

[23] Q. Zhang, C. Hu, S. Upasani *et al.*, "Agentic context engineering: Evolving contexts for self-improving language models," arXiv preprint arXiv:2510.04618, 2025.

[24] W. Zhou, Y. Ou, S. Ding, L. Li, J. Wu, T. Wang, J. Chen, S. Wang, X. Xu, N. Zhang, H. Chen, and Y. E. Jiang, "Symbolic learning enables self-evolving agents," *arXiv preprint arXiv:2406.18532*, 2024.

[25] E. Zelikman, E. Lorch, L. Mackey, and A. T. Kalai, "Self-taught optimizer (STOP): Recursively self-improving code generation," in *First Conference on Language Modeling (COLM 2024)*, 2024, arXiv:2310.02304.

[26] M. Robeyns, M. Szummer, and L. Aitchison, "A self-improving coding agent," *arXiv preprint arXiv:2504.15228*, 2025.

[27] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS 2023), Datasets and Benchmarks Track*, 2023, arXiv:2306.05685.

[28] E. Palumbo, A. Tamborrino, V. Ode, B. Lacker, A. Casas Escoda, J. Hopple, M. Better, J. Leoni, H. Galvão, H. Bouchard, M. Lalmas, J. L. Redondo García, A. Abebe, A. Clifton, A. Blomberg, H. Lindström, D. Doro, and C. Doig Cardet, "Bootstrapping conversational recommendation agents at spotify: Synthetic data generation and self-improvement loops," *arXiv preprint arXiv:2609.30297*, 2026.

[29] J. Zhang *et al.*, "Darwin Gödel machine: Open-ended evolution of self-improving agents," *arXiv preprint*, 2025.

[30] A. Novikov *et al.*, "AlphaEvolve: A coding agent for scientific and algorithmic discovery," arXiv preprint arXiv:2506.13131, 2025.

[31] Y. Yang, Z. Gong, W. Huang, Q. Yang, Z. Zhou, Z. Huang, Y. Li, X. Gao, Q. Dai, B. Liu, K. Qiu, Y. Yang, D. Chen, X. Yang, and C. Luo, "SkillOpt: Executive strategy for self-evolving agent skills," arXiv preprint arXiv:2605.23904, 2026. [Online]. Available: https://microsoft.github.io/SkillOpt

[32] Cognition, "Devin's 2025 performance review: Learnings from 18 months of agents at work," https://cognition.com/blog/devin-annual-performance-review-2025, 2025.

[33] J. Wang *et al.*, "Reinforcement learning for self-improving agent with skill library," *arXiv preprint*, 2025.

[34] P. Xia *et al.*, "SkillRL: Evolving agents via recursive skill-augmented reinforcement learning," *arXiv preprint*, 2026.

[35] C. Ling, P. Chen, A. Guan, J. Qu, S. A. Akbar, M. Gopinathan, and E. Cornejo, "PACE: Two-timescale self-evolution for small language model agents," *arXiv preprint*, 2026.

[36] X. Liu *et al.*, "SkillForge: Forging domain-specific, self-evolving agent skills in cloud technical support," *arXiv preprint*, 2026, accepted at ACM SIGIR 2026 Industry Track.

[37] X. Huang *et al.*, "CASCADE: Cumulative agentic skill creation through autonomous development and evolution," *arXiv preprint*, 2025.

[38] L. Gao, J. Schulman, and J. Hilton, "Scaling laws for reward model overoptimization," *arXiv preprint arXiv:2210.10760*, 2022.

[39] S. Shankar, J. Zamfirescu-Pereira, B. Hartmann, A. G. Parameswaran, and I. Arawjo, "Who validates the validators? aligning LLM-assisted evaluation of LLM outputs with human preferences," *arXiv preprint arXiv:2404.12272*, 2024.

[40] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha, "The AI scientist: Towards fully automated open-ended scientific discovery," *arXiv preprint arXiv:2408.06292*, 2024.

[41] J. Xu *et al.*, "NanoResearch: Co-evolving skills, memory, and policy for personalized research automation," *arXiv preprint*, 2026.

[42] A. Qu, H. Zheng, Z. Zhou, Y. Yan, Y. Tang, S. Y. Ong, F. Hong, K. Zhou, C. Jiang, M. Kong, J. Zhu, X. Jiang, S. Li, C. Wu, B. K. H. Low, J. Zhao, and P. P. Liang, "CORAL: Towards autonomous multi-agent evolution for open-ended discovery," *arXiv preprint*, 2026.

[43] K. T.-i. Ong, M. Kang, D. Choi, J. Cho, S. Kim, S. Lim, G. Jang, M. Oh, B. Jeong, S. Kim, T. Kwon, and J. Yeo, "Towards direct evaluation of harness optimizers via priority ranking," *arXiv preprint*, 2026.

[44] A. Gunjal, A. Wang, E. Lau, V. Nath, Y. He, B. Liu, and S. Hendryx, "Rubrics as rewards: Reinforcement learning beyond verifiable domains," *arXiv preprint arXiv:2507.17746*, 2025.

[45] R. Jia, Y. Yang, Y. Gai, K. Luo, S. Huang, J. Lin, X. Jiang, and G. Jiang, "Writing-zero: Bridge the gap between non-verifiable tasks and verifiable rewards," *arXiv preprint arXiv:2506.00103*, 2025.

[46] K. Shi, Y. Liu, P. Wang, A. R. Fabbri, S. Joty, and A. Cohan, "References improve LLM alignment in non-verifiable domains," *arXiv preprint arXiv:2602.16802*, Feb. 2026.

[47] X. Jin, J. Zhang, T. Cao, and B. Yang, "Beyond score prediction: LLM-based essay scoring and feedback generation via reinforcement learning with rubric rewards," *arXiv preprint arXiv:2607.19219*, Jul. 2026.

[48] T. Yu, B. Ji, S. Wang, S. Yao, Z. Wang, G. Cui, L. Yuan, N. Ding, Y. Yao, Z. Liu, M. Sun, and T.-S. Chua, "RLPR: Extrapolating RLVR to general domains without verifiers," *arXiv preprint arXiv:2506.18254*, Jun. 2025.

[49] R. Xu, T. Liu, Z. Dong, T. Yu, I. Hong, C. Yang, L. Zhang, T. Zhao, and H. Wang, "Alternating reinforcement learning for rubric-based reward modeling in non-verifiable LLM post-training," *arXiv preprint arXiv:2602.01511*, 2026.

[50] H. Ding, B. Huang, Y. Fang, W. Liao, Z. Li, J. Zhang, Z. Wu, J. Zhao, and Y. Wang, "EvoRubrics: Dynamic rubrics as rewards via adversarial co-evolution for LLM reinforcement learning," *arXiv preprint arXiv:2606.23038*, Jun. 2026.

[51] H. Zhu, C. Cai, Y. Song, X. Chen, S. Han, and Y. Guo, "Self-evolving deep research via joint generation and evaluation," *arXiv preprint arXiv:2606.04507*, Jun. 2026.

[52] G. Li, B. Dalvi Mishra, Z. Wang, J. Yan, Y. Chen, C.-L. Li, L. T. Le, R. Han, G. Lee, H. Tong, C.-Y. Lee, and T. Pfister, "RubricEM: Meta-RL with rubric-guided policy decomposition beyond verifiable rewards," *arXiv preprint arXiv:2605.10899*, May 2026.

[53] Y. Hu, M. Xu, H. Guo, Y. Song, X. Zhu, and B. Zheng, "Skillchain: Closing the loop on skill evolution for image-based e-commerce ai assistants," *arXiv preprint arXiv:2606.12984*, 2026.

[54] J. Zhu, K. Mao, Y. Guo, D. He, S. Xu, S. Gu, and Y. Yue, "Skillcoach: Self-evolving rubrics for evaluating and enhancing agentic skill-use," *arXiv preprint arXiv:2607.01874*, 2026.

[55] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, "Cognitive architectures for language agents (CoALA)," *Transactions on Machine Learning Research (TMLR)*, 2023.

[56] AGENTS.md, "AGENTS.md: A cross-vendor open standard for agent instructions," 2025. [Online]. Available: https://agents.md

[57] R. Robbes, T. Matricon, T. Degueule, A. Hora, and S. Zacchiroli, "Agentic much? adoption of coding agents on github," *arXiv preprint arXiv:2601.18341*, 2026.

[58] M. Galster, S. Mohsenimofidi, J. L. Lulla, M. A. Abubakar, C. Treude, and S. Baltes, "Harness engineering for agentic ai coding tools: An exploratory study," *arXiv preprint arXiv:2602.14690*, 2026.

[59] T. Gloaguen, N. Mündler-Sasahara, M. N. Müller, V. Raychev, and M. Vechev, "Evaluating agents.md: Are repository-level context files helpful for coding agents?" *arXiv preprint arXiv:2602.11988*, 2026.

[60] X. Wu *et al.*, "Co-evolving LLM decision and skill bank agents for long-horizon tasks," *arXiv preprint*, 2026.

[61] X. Zhang, Y. Cui, G. Wang, Z. Li, W. Qiu, B. Zhu, and P. He, "Library drift: Diagnosing and fixing a silent failure mode in self-evolving llm skill libraries," *arXiv preprint arXiv:2605.19576*, 2026.

[62] Y. Wei, Z. Huang, S. Lu, J. Qian, Q. Wang, C. Wu, and L. He, "Skillsmith: Co-evolving skills and tools for self-improving agent systems," *arXiv preprint arXiv:2606.01314*, 2026.

[63] H. Pu, X. Song, and L. Zhao, "Skillops: Managing llm agent skill libraries as self-maintaining software ecosystems," *arXiv preprint arXiv:2605.13716*, 2026.

[64] Y. Wang, Y. Zhou, Y. Liang, C. Zhang, F. Liu, J. Zhou, and H. Yao, "Not all skills help: Measuring and repairing agent knowledge," *arXiv preprint arXiv:2606.15390*, 2026.

[65] C. Shen and E. Hruschka, "Who maintains agent skills? A longitudinal study of human-governed, AI-assisted skill maintenance," *arXiv preprint arXiv:2609.05677*, 2026.

[66] M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, and J. Schmidhuber, "Language agents as optimizable graphs," in *Forty-first International Conference on Machine Learning (ICML 2024)*, 2024, arXiv:2402.16823.

[67] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Zheng, B. Liu, Y. Luo, and C. Wu, "AFlow: Automating agentic workflow generation," *arXiv preprint arXiv:2410.10762*, 2024.

[68] Y. Cheng, C. Liu, H. Yu, H. Tan, T. Wu, W. Jin, J. Ran, K. Wang, X. Li, H. Bai, G. Chesi, and N. Wong, "Optimas: Automatically optimize multi-agent system," *arXiv preprint arXiv:2608.21918*, 2026.

[69] J. Chu, J. Seo, J. Cho, J. Na, Y. Xiong, Y. Kim, and H. J. Kim, "Agentgrad: Intervention-guided prompt optimization for multi agent systems," *arXiv preprint arXiv:2609.08572*, 2026.

[70] M. Chen, C. Lv, G. Zhang, H. Chang, and S. Zhou, "Harnessforge: Joint harness and policy evolution for adaptive agent systems," *arXiv preprint arXiv:2606.01779*, 2026.

[71] S. Zhao, R. Kong, Z. Zhang, L. Shen, X. He, X. Zhang, and Q. Zhang, "Topological coherence for self-evolving multi-agent systems," *arXiv preprint arXiv:2609.37953*, 2026.

[72] R. Wang and J. Cui, "Evo2team: When do evolved skills transfer? from selection to deployment," *arXiv preprint arXiv:2609.34135*, 2026.

[73] Letta, "MemFS; memory & dreaming; introducing context repositories: Git-based memory for coding agents," https://docs.letta.com/letta-code/memfs, https://docs.letta.com/letta-agent/memory, https://www.letta.com/blog/context-repositories/, 2026.

[74] Mem0, "Mem0 platform: Memory history, feedback, and changelog," https://docs.mem0.ai/api-reference/memory/history-memory, https://docs.mem0.ai/api-reference/memory/feedback, https://docs.mem0.ai/changelog, 2026.

[75] Zep, "Zep documentation: Facts; february 2026 deprecation wave; changelog," https://help.getzep.com/facts, https://help.getzep.com/february-2026-deprecation-wave, https://help.getzep.com/changelog/2026/9/24, 2026.

[76] Z. Wang, "TOKI: A bitemporal operator algebra for contradiction resolution in LLM-agent persistent memory," *arXiv preprint arXiv:2606.06240*, Jun. 2026.

[77] J. Xu, Y. Xiao, W. Shao, H. Liu, and X. Li, "Memory provenance laundering in LLM agents: A non-amplification firewall for persistent memory," *arXiv preprint arXiv:2607.29167*, Jul. 2026.

[78] Y. Margalit, N. Cohen-Inger, E. Avram, R. Taig, and O. Margalit, "Governed shared memory for multi-agent LLM systems," *arXiv preprint arXiv:2606.24535*, Jun. 2026.

[79] Z. Yu, Y. Wu, Z. Yin, K. Chen, Z. Zhao, M. Wang, S. Yan, and L. Yang, "Recursive experiential–working memory evolution for long-horizon agent harnesses," *arXiv preprint arXiv:2608.24876*, Aug. 2026.

[80] H. Wang, G. Dong, H. Liang, Z. Zhang, J. Luo, C. Liu, C. Xue, and H. Tang, "MemGuard: Persisting verifier signals for LLM-agent memory governance," *arXiv preprint arXiv:2608.21867*, Aug. 2026.

[81] X. Sun, Q. Wang, H. Li, and L. Chen, "Verifiable memory: Learning unified memory management with local and global verifiers for large language model agents," *arXiv preprint arXiv:2608.03137*, Aug. 2026.

[82] agentic-community, "Mcp gateway & registry," https://github.com/agentic-community/mcp-gateway-registry, 2026.

[83] Microsoft, "Inventory and discover mcp servers in your api center," https://learn.microsoft.com/en-us/azure/api-center/register-discover-mcp-server, 2026.

[84] TrueFoundry, "Mcp gateway authentication and security," https://www.truefoundry.com/docs/ai-gateway/mcp-gateway-auth-security, 2026.

[85] R. Kang and V. Wang, "Registry-governed agent lifecycle: Completing EDDOps with evaluation-driven registration, promotion, and retirement on AWS AgentCore," *arXiv preprint arXiv:2607.00345*, Jul. 2026.

[86] A. Quessada-Vial, "Agentic configuration management (ACM): A reference configuration model for governed agentic systems," *arXiv preprint arXiv:2608.11166*, Aug. 2026.

[87] X. Qin, S. Luan, J. See, Z. Boukhers, C. Yang, and Z. Li, "Governed capability evolution: Lifecycle-time compatibility checking and rollback for AI-component-based systems, with a proof-of-concept evaluation on embodied agents," *arXiv preprint arXiv:2604.08059*, Apr. 2026.

[88] M. Helwig, "Memory as infrastructure: Reliability engineering for persistent agent memory in months-long LLM-assisted development," *arXiv preprint arXiv:2609.05510*, Aug. 2026.

[89] J. Hu, H. Zhao, Q. Jia, D. Chen, T. Zhao, S. Duan, and G. Liu, "Learning from others, acting for you: Cross-user memory sharing for llm agents," *arXiv preprint arXiv:2609.32511*, 2026.

[90] J. Nie, Y. Zhang, Q. Cai, Y.-m. Cheung, X. Tian, and B. Han, "Evolvenet: Collaborative harness evolution for agent self-improvement," *arXiv preprint arXiv:2608.04968*, 2026.

[91] Y. Guo, H. Zhou, J. Zhang, Y. Zhang, Y. Liu, Q. Chen, Q. Lin, H. Dong, J. M. Zhang, and Z. Chen, "Xreposkill: Learning transferable skills for software engineering agents," *arXiv preprint arXiv:2609.36807*, 2026.

[92] J. Chu, J. S. Lee, J. Park, D. Ko, J. Na, S. Lee, T. Lee, M. Yoon, M. Joo, Y. Xiong, and H. J. Kim, "Retrieval-augmented skill optimization via cross-harness adaptation," *arXiv preprint arXiv:2609.38024*, 2026.

[93] Y. T. Shen, K. Toyoda, and A. Leung, "Revoked but still authoritative: An empirical study of revocation enforcement in agent-memory systems," *arXiv preprint arXiv:2609.08258*, Sep. 2026.

[94] A. Miyai, K. Aizawa, and T. Yamasaki, "Task-coevolve: Efficient harness optimization via adaptive validation task selection," *arXiv preprint arXiv:2608.20169*, 2026.

[95] S. Park, W. Kim, R. Tan, J. Zhang, W.-S. Han, P. Gao, C. Park, Y. Yao, R. Fu, E. Nallipogu, Q. Lin, S. Rajmohan, and D. Zhang, "Autosaddler: Automatic harness optimization with durable updates from agent execution traces," *arXiv preprint arXiv:2608.23041*, 2026.

[96] J. Qi, Y. He, J. Liu, Q. Huang, S. Hu, Z. Dang, G. Chen, R. Yang, W. Jiang, Y. Liu, T. Lyu, and F. Li, "Vace: Validation-gated alternating co-evolution of agent models and harnesses," *arXiv preprint arXiv:2609.37105*, 2026.

[97] Q. Yang, Z. Kou, X. Zhang, D.-D. Wu, H. Gu, J. Guo, Y. Liu, D. Jiang, and Q. Xu, "Federated agent optimization," *arXiv preprint arXiv:2610.01195*, 2026.

[98] X. Wang *et al.*, "AutoAgent: Evolving cognition and elastic memory orchestration for adaptive agents," *arXiv preprint*, 2026.

[99] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi, "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, pp. 468–475, 2024.

[100] S. Hu, C. Lu, and J. Clune, "Automated design of agentic systems (ADAS)," *arXiv preprint*, 2024.

[101] G. Wang *et al.*, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint*, 2023.

[102] Sierra, "Agent OS 2.0: From answers to memory and action," 2025. [Online]. Available: https://sierra.ai/blog/agent-os-2-0

[103] Braintrust, "Use prompts in code," Braintrust documentation, https://www.braintrust.dev/docs/guides/prompts, 2026, accessed 2026-10-03.

[104] Vellum, "Release tags," Vellum documentation, https://docs.vellum.ai/product/deployments/release-tags, 2026, accessed 2026-10-03.

[105] LangChain, "Prompt engineering concepts," LangSmith documentation, https://docs.langchain.com/langsmith/prompt-engineering-concepts, 2026, accessed 2026-10-03.

[106] P. Barbaste, T. Darrigol, G. Vu, and T. Wiltberger, "Harness engineering: Anatomy, architecture, and evolution of coding agents – a source-code study of eleven systems," *arXiv preprint arXiv:2609.00006*, 2026.

[107] J. L. Lulla, V. Nersesyan, S. Mohsenimofidi, C. Treude, and S. Baltes, "Loop engineering: Building blocks, adoption, and impact," *arXiv preprint arXiv:2608.21884*, 2026.

[108] Z. Jiang and B. Dong, "Buildrix: An open platform for sharing and benchmarking agentic ai skills in building engineering," *arXiv preprint arXiv:2606.25139*, 2026.

[109] T. Q. Nguyen, S. Desai, R. H. Anwar, F. Shaik, V. Suryanarayanan, and V. Chowdhary, "VerificAgent: Domain-specific memory verification for scalable oversight of aligned computer-use agents," *arXiv preprint arXiv:2506.02539*, Jun. 2025.

[110] Y. Wang, L. Xia, R. Liu, Z. Zhang, H. Wu, Y. Yang, J. He, Y. Guo, and K. Lei, "Sage: A statistical acceptance gate for self-evolving agents," *arXiv preprint arXiv:2609.36043*, 2026.

[111] L. Shang, M. Xu, Y. Sun, T. Xia, L. Hu, L. Xu, and N. Zheng, "When self-evolution backfires: Pre-commit gating against skill contamination in llm agents," *arXiv preprint arXiv:2608.05810*, 2026.

[112] Y. Shen, B. Li, and X. Zhang, "SkillOpt-Lite: Better and faster agent self-evolution via one line of vibe," *arXiv preprint arXiv:2607.03451*, 2026.

[113] L. A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. G. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab, "GEPA: Reflective prompt evolution can outperform reinforcement learning," *arXiv preprint*, 2025.

[114] N. Vassilyev, W. Berrios, R. Zhang, B. Han, D. Kiela, and S. Mehri, "Reflective context learning: Studying the optimization primitives of context space," *arXiv preprint*, 2026.

[115] Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab, and C. Finn, "Meta-Harness: End-to-end optimization of model harnesses," *arXiv preprint*, 2026.

[116] J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, and Y.-G. Jiang, "Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses," *arXiv preprint*, 2026.

[117] Q. Cai, Y. Zhang, X. Jia, H. Zheng, W. Xue, J. Song, X. Tian, and Y. Guo, "MOSS: Self-evolution through source-level rewriting in autonomous agent systems," *arXiv preprint*, 2026.
[118] T. Zehle, "CANTANTE: Optimizing agentic systems via contrastive credit attribution," *arXiv preprint*, 2026.
[119] M. Nguyen, W. C. Tan, N. A. Hassan, A. Raman, L. H. Lim, and A. F. Razak, "Where does harness-optimization value live? Localized gains and the budget-splitting trap in self-evolving LLM agents," *arXiv preprint arXiv:2609.02889*, 2026.
[120] L. Li, J. Li, J. Zhao, K. Ye, Y. Li, C.-z. Xu, and X. Gao, "Grow the harness, not the context: From strategy-free scaffolds to reusable specialist agents," *arXiv preprint arXiv:2609.26760*, 2026.
[121] X. Zhang, Y. Cui, G. Wang, Z. Li, W. Qiu, B. Zhu, and P. He, "Ratchet: A minimal hygiene recipe for self-evolving LLM agents," *arXiv preprint*, 2026.
[122] M. M. Tanjim, J. Subramanian, X. Chen, B. Kveton, S. Mukherjee, A. Zhang, S. Kim, S. Sarkhel, and S. Choudhury, "MOCHA: Multi-objective chebyshev annealing for agent skill optimization," *arXiv preprint*, 2026.
[123] H. Liu, H. Yang, T. Jiang, B. Tang, F. Xiong, Y. Luo, and Z. Li, "SkillsVote: Lifecycle governance of agent skills from collection, recommendation to evolution," *arXiv preprint arXiv:2605.18401*, 2026.
[124] Y. Shalev, Z. Ding, and M. Jamnik, "Training language agents to learn from experience," *arXiv preprint*, 2026.
[125] H. Li, R. He, Q. Zhang, C. Ji, Q. Mang, X. Chen, L. A. Agrawal, W.-L. Liao, E. Yang, A. Cheung, J. Zou, K. Olukotun, I. Stoica, and J. E. Gonzalez, "Combee: Scaling prompt learning for self-improving language model agents," *arXiv preprint*, 2026.
[126] J. Yang, G. Yao, Y. Zhang, R. R. Kompella, G. Liu, and S. Chang, "FederatedSkill: Federated learning for agentic skill evolution," *arXiv preprint arXiv:2606.03143*, 2026.
[127] P. Hebbar, Y. Manawat, S. Verboomen, A. Ivanova, S. Palanimalai, K. Bhatia, and V. Baskaran, "Sia: Self improving ai with harness & weight updates," *arXiv preprint arXiv:2605.27276*, 2026.
[128] H. Gao, H. Chen, C. Wang, S. Guo, L. Pang, Z. Liu, H. Shen, and X. Cheng, "Skillaudit: Ground-truth-free skill evolution via paired trajectory auditing," *arXiv preprint arXiv:2606.14239*, 2026.
[129] T. Sah, D. Sah, H. Jain, and T. Sah, "Evoundo: Recoverability-constrained self-evolution for llm agent harnesses," *arXiv preprint arXiv:2608.28363*, 2026.
[130] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 54, 2017, pp. 1273–1282.
[131] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private recurrent language models," in *International Conference on Learning Representations (ICLR)*, 2018, arXiv:1710.06963.
[132] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis *et al.*, "Advances and open problems in federated learning," *arXiv preprint arXiv:1912.04977*, 2019, published in Foundations and Trends in Machine Learning, vol. 14, no. 1–2, 2021.
[133] T. Guo, S. Guo, J. Wang, and W. Xu, "PromptFL: Let federated participants cooperatively learn prompts instead of models – federated learning in age of foundation model," *arXiv preprint arXiv:2208.11625*, 2022.
[134] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, 2015.
[135] D. Jaroslawicz *et al.*, "How many instructions can LLMs follow at once?" *arXiv preprint*, 2025.
[136] W. Zou *et al.*, "PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models," *arXiv preprint*, 2024, accepted at USENIX Security 2025.
[137] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases," *arXiv preprint arXiv:2407.12784*, 2024.
[138] Z. Xu *et al.*, "From storage to steering: Memory control flow attacks on LLM agents," *arXiv preprint*, 2026.
[139] Y. Su, W. Xu, C. Zuo, and E. Bertino, "ChronoMem: Version control and semantic rollback for large language model agent memory," *arXiv preprint arXiv:2607.27773*, 2026.
[140] C. Yu, Y. Wang, J. Zhang, Y. Duan, M. Zheng, Z. Wu, K. Shi, and T. Cai, "From faulty memories to corrected actions: Dependency-guided rollback repair for memory-augmented agents," *arXiv preprint arXiv:2608.10502*, 2026.
[141] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," *arXiv preprint arXiv:2302.12173*, 2023.
[142] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," *arXiv preprint arXiv:2406.13352*, 2024.
[143] Check Point Research, "MCPoison: Cursor IDE's MCP vulnerability (CVE-2025-54136)," https://research.checkpoint.com/2025/cursor-vulnerability-mcpoison/, 2025.
[144] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," *arXiv preprint arXiv:2404.13208*, 2024.
[145] S. Singh *et al.*, "The leaderboard illusion," *arXiv preprint*, 2025.
[146] M. Cemri *et al.*, "Why do multi-agent LLM systems fail?" *arXiv preprint*, 2025.
[147] N. F. Liu *et al.*, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics (TACL)*, 2023.

MIKHAIL L. ARBUZOV TODO(bio).

LEE MOSBACKER TODO(bio).

SISONG BEI TODO(bio).

ZIWEI DONG TODO(bio).

DMITRI KALAEV TODO(bio).

ALEXEY SHVETS TODO(bio).